

0 — Enterprise RAG

Design Document

Naresh Chaurasia

Version 1.0, 2026-07-14: Initial enterprise design — converting Company Knowledge Assistant (POC) to an enterprise-wide, AWS-native, security/compliance-first RAG platform

Table of Contents

1. 1. Purpose & Scope	2
2. 2. Current State Assessment (POC → Gaps)	3
3. 3. Enterprise Requirements	6
3.1. 3.1 Functional Requirements	6
3.2. 3.2 Non-Functional Requirements	6
4. 4. Target Architecture (AWS-Native)	7
4.1. 4.1 High-Level Architecture Diagram	7
4.2. 4.2 Component Mapping (POC → Enterprise)	9
4.3. 4.3 Networking	10
5. 5. Security & Compliance (Primary Focus).....	11
5.1. 5.1 Authentication & Authorization	11
5.2. 5.2 Encryption	11
5.3. 5.3 Secrets Management	12
5.4. 5.4 PII & Data Governance	12
5.5. 5.5 Audit Logging	13
5.6. 5.6 LLM-Specific Security	13
5.7. 5.7 Compliance Control Mapping (Illustrative)	13
6. 6. Scalability & Reliability	15
7. 7. Observability	16
8. 8. Data & Ingestion Governance	17
9. 9. CI/CD & Environment Strategy	18
10. 10. Migration Roadmap	19
11. 11. Cost Management	20
12. 12. MLOps / LLMOps	21
12.1. 12.1 Model & Prompt Governance	21
12.2. 12.2 Evaluation & Continuous Quality Monitoring	21
12.3. 12.3 Guardrails	22
12.4. 12.4 LLMOps Tooling Summary	22
13. 13. Extension Notes: Future Multi-Tenancy	24
14. 14. Open Questions / Risks	25



Document Type: Technical Design Document (Level 400 — Deep Technical / Architect & Engineering Audience)

What this document covers:

- The end-to-end target architecture required to convert the existing Company Knowledge Assistant RAG proof-of-concept into an AWS-native, enterprise-grade production system.
- Security and compliance controls.
- Scalability and observability.
- CI/CD.
- MLOps/LLMOps.
- Migration plan.

Intended audience: Engineers, architects, security/compliance reviewers, and platform/DevOps teams who will design, build, review, and operate the system.

Chapter 1. 1. Purpose & Scope

This document defines the target architecture and design decisions required to convert the existing **Company Knowledge Assistant** RAG proof-of-concept (documented in [1-rag-knowledge-management.adoc](#) and [2-rag-knowledge-management.adoc](#)) into an **enterprise-wide, production-grade system**.

Attribute	Decision
Deployment target	AWS-native (EKS, RDS, ElastiCache, S3, Secrets Manager, KMS, CloudWatch/CloudTrail). Cloud-agnostic patterns (containers, IaC) are used so the design could be ported to Azure/GCP with moderate effort.
Tenancy model	Single-tenant enterprise deployment. One organization, one production environment per region. Focus is on scale, high availability, and governance — not multi-org data isolation. (Section 12 notes how the design could evolve to multi-tenant if required later.)
Primary design priority	Security & Compliance. Authentication/authorization, encryption, secrets management, PII/data governance, and auditability are treated as first-class, non-negotiable requirements. Scalability, observability, CI/CD, and cost are covered at standard enterprise depth.
Non-goals (this revision)	Multi-tenant data isolation, multi-cloud abstraction layer, fine-tuning custom LLMs, real-time streaming ingestion (batch/near-real-time ingestion is in scope).

This document is a **design/architecture document**, not an implementation runbook. Implementation runbooks (local setup, Docker Compose) remain in [1-rag-knowledge-management.adoc](#); the current-state file-by-file explanation remains in [2-rag-knowledge-management.adoc](#).

Chapter 2. 2. Current State Assessment (POC → Gaps)

The existing implementation is a single-machine / single Docker-Compose stack: FastAPI (`app/api.py`) + LangChain RAG chain (`app/rag.py`) + PostgreSQL/pgvector (`app/utls.py`) + optional Redis semantic cache + Cohere reranker + OpenAI embeddings/LLM.

The table below maps concrete gaps observed in the code to the enterprise requirement they violate.

Area	Current State (POC)	Enterprise Gap
AuthN/AuthZ	No authentication on any endpoint. <code>/ask</code> , <code>/ingest</code> , <code>/ingest/status</code> are open to anyone who can reach port 8000.	No user identity, no RBAC, no way to enforce "who can ask/ingest/admin." Any client can trigger expensive LLM calls or re-ingest data.
Secrets management	<code>.env</code> file with plaintext <code>OPENAI_API_KEY</code> , <code>CO_API_KEY</code> , <code>DATABASE_URL</code> (including password), committed/shipped alongside the app; loaded via <code>dotenv</code> (<code>app/utls.py</code>) and <code>env_file</code> in <code>docker-compose.yml</code> .	Secrets in <code>.env</code> /version control violate least-privilege and rotation requirements. No centralized secret store, no rotation, no access audit trail on secret reads.
Encryption in transit	App communicates with OpenAI/Cohere over TLS (library default), but internal traffic (Postgres, Redis) runs unencrypted on a Docker bridge network; no mTLS between services.	Enterprise requires TLS/mTLS for all service-to-service traffic, including DB and cache connections, not just external API calls.
Encryption at rest	Postgres data directory and Redis AOF persisted to local Docker named volumes (<code>pgdata</code> , <code>redis-data</code>) with no volume-level encryption.	No KMS-backed encryption at rest for vectors, document metadata, or cached responses (which may contain sensitive answer text).
Network exposure	<code>docker-compose.yml</code> publishes Postgres (5432), Redis (6379/8001), and the app (8000) directly on host ports with no network segmentation.	Database and cache should never be reachable from outside a private subnet; no WAF, no ALB, no security-group-based segmentation.

Area	Current State (POC)	Enterprise Gap
PII / data governance	<code>app/ingest.py</code> ingests any file placed under <code>data/<category>/</code> verbatim — no scanning for PII, no classification, no redaction before embedding/storage.	Enterprise documents (HR handbooks, expense reports, PTO policies) may contain PII (names, salaries, bank details). No control exists to detect, tag, or redact PII before it becomes part of a vector index that any authenticated user with category access could retrieve verbatim.
Audit logging	<code>app/logger.py</code> provides local file/console logging (<code>logs/app.log</code>) of request timing and internal steps; no immutable audit trail of who asked what , what was retrieved , or admin actions (e.g., who triggered ingestion).	Compliance (SOC 2, internal audit, incident response) requires a tamper-evident audit log of user identity, question, retrieved sources, and answer — retained per policy.
Access control on content	<code>/ask</code> category filtering exists in code (<code>app/rag.py, _build_chain(category)</code>) but <code>app/api.py</code> hard-codes <code>category = None</code> (no filter applied) — meaning any user can retrieve content from any category , including HR/handbook/policy documents.	No mapping between user identity/role and the categories/documents they are authorized to query. This is a direct data-governance and least-privilege gap.
High availability	Single Postgres container, single Redis container, single app container (or <code>--reload</code> dev server). No replicas, no failover.	No RTO/RPO guarantees; a single container/node failure causes full outage and, without backups, potential permanent data loss of the vector index.
Backup & DR	No automated backup strategy for the <code>langchain_pg_embedding</code> table or ingested source documents beyond what's on local disk (<code>data/</code>).	No cross-region backup, no point-in-time recovery, no tested restore procedure.
Rate limiting / cost control	No request throttling on <code>/ask</code> (each call invokes OpenAI embeddings + GPT-4o-mini + Cohere rerank). Semantic caching is explicitly disabled in <code>app/rag.py</code> (<code>set_llm_cache(None)</code>) due to a correctness bug (over-permissive <code>distance_threshold=0.98</code>).	Uncontrolled cost exposure; no per-user/per-department quota; the caching bug needs a proper fix (stricter threshold + validation) rather than permanent disablement, or a safer alternative (deterministic cache keyed on normalized query + category).

Area	Current State (POC)	Enterprise Gap
Dependency management	<code>requirements.txt</code> referenced by <code>Dockerfile</code> is documented as missing from the repo (see <code>2-rag-knowledge-management.adoc</code> , §6/§8); actual pinned versions live scattered across <code>ALL-ENV/*.txt</code> files at the repo root.	No single source of truth for dependency versions; no vulnerability scanning (SCA) of the dependency tree; no reproducible builds.
Observability	Local Python logging only; no metrics, no tracing (LangSmith is configured but optional/env-gated), no alerting.	No SLO dashboards, no automated alerting on error rate/latency/cost anomalies, no distributed tracing across retrieval → rerank → generation.
CI/CD	No pipeline definitions found in the project; deployment is manual (<code>docker compose up --build</code> or local <code>uvicorn</code>).	No automated testing, image scanning, or promotion gates between dev/staging/prod.
Data residency / retention	No documented retention policy for ingested documents, vector embeddings, logs, or cached answers.	Regulatory frameworks (GDPR, internal records-retention policy) require explicit retention and deletion (“right to erasure”) capability.

Chapter 3. 3. Enterprise Requirements

3.1. 3.1 Functional Requirements

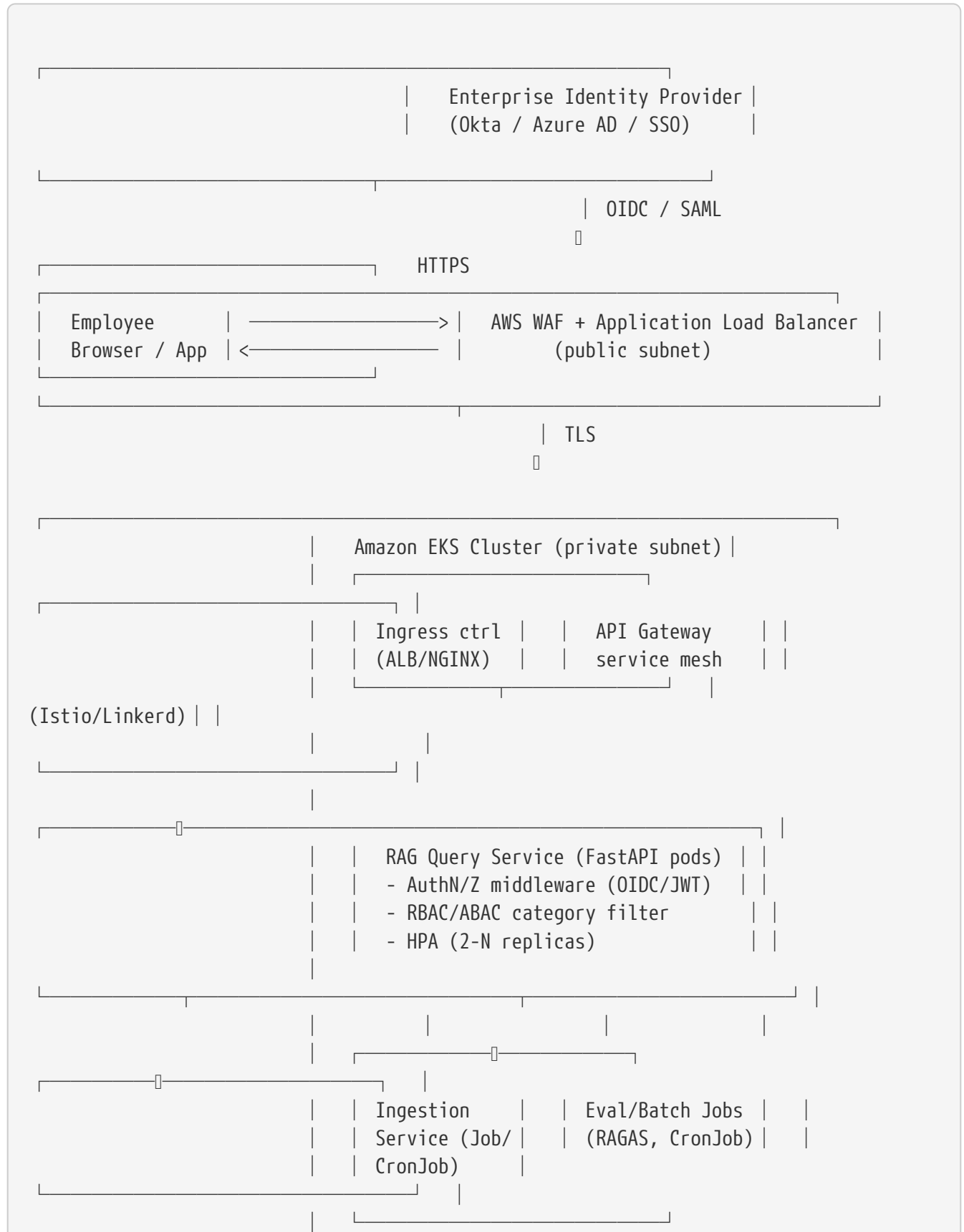
- Employees across the enterprise (all departments/regions) can ask natural-language questions and receive grounded answers sourced from authorized internal documents only.
- Content owners/admins can ingest, update, version, and retire documents through a controlled workflow (not ad-hoc file drops).
- Access to answers/sources is scoped by the requesting user's role and department (category-based ABAC/RBAC), replacing the current hard-coded `category=None`.
- All administrative actions (ingestion trigger, index rebuild, config change) require elevated privileges and are fully audited.

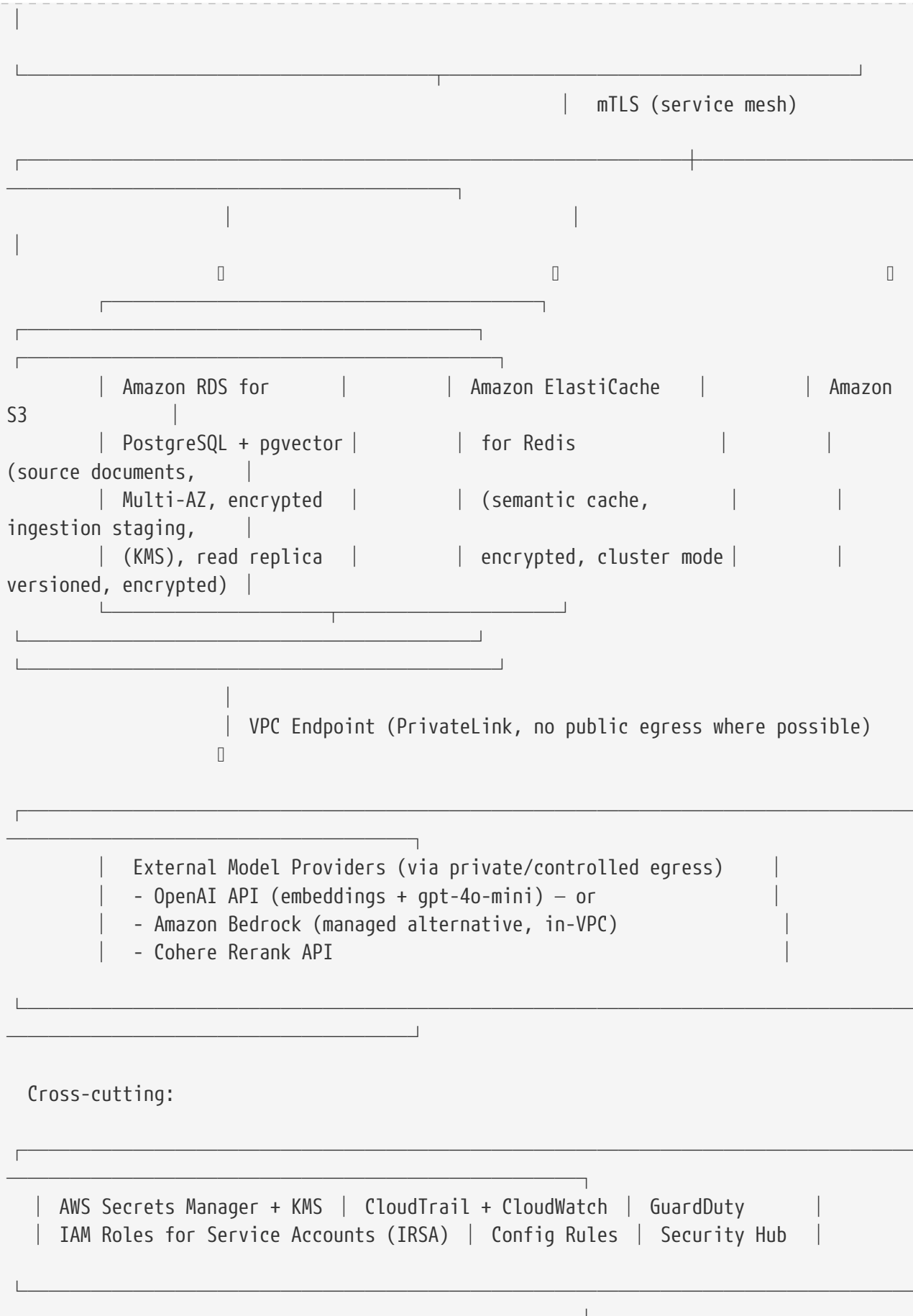
3.2. 3.2 Non-Functional Requirements

Requirement	Target
Availability	99.9% monthly uptime for the query path (<code>/ask</code>); ingestion path may have a lower SLA (99.0%) since it is background/batch.
Recovery Point Objective (RPO)	≤ 15 minutes for the vector database (via automated snapshots + WAL/point-in-time recovery).
Recovery Time Objective (RTO)	≤ 1 hour for full-region failover in a single-region deployment; ≤ 4 hours for cross-region DR restore.
Latency	p95 end-to-end <code>/ask</code> response ≤ 4 seconds (embedding + vector search + rerank + LLM generation), excluding cold starts.
Compliance	SOC 2 Type II control alignment, GDPR (for any EU employee/customer PII), internal records-retention policy, least-privilege IAM (CIS AWS Foundations Benchmark).
Auditability	100% of <code>/ask</code> and admin requests logged immutably with user identity, timestamp, inputs, and outputs; retained ≥ 1 year (configurable per policy).
Scalability	Support horizontal scale-out of the query service to N replicas under load; vector store sized for millions of chunks across all enterprise document categories.
Data residency	All data (documents, embeddings, logs, cache) stored within the enterprise's approved AWS region(s); no data sent to third-party LLM providers outside contractual data-processing agreements.

Chapter 4. 4. Target Architecture (AWS-Native)

4.1. 4.1 High-Level Architecture Diagram





4.2. 4.2 Component Mapping (POC → Enterprise)

POC Component (current code)	Enterprise Replacement	Notes
Docker Compose (<code>docker-compose.yml</code>)	Amazon EKS (Kubernetes) + Helm charts	Same container images (<code>Dockerfile</code>), orchestrated declaratively; supports rolling updates, HPA, and pod-level network policies.
<code>pgvector/pgvector:pg17</code> container	Amazon RDS for PostgreSQL (pgvector extension enabled) — Multi-AZ, encrypted, automated backups, read replica for eval/reporting workloads	<code>app/utils.py</code> 's <code>'PGEngine.from_connection_string()</code> continues to work unchanged; only <code>DATABASE_URL</code> changes (pulled from Secrets Manager, not <code>.env</code>).
<code>redis/redis-stack:latest</code> container	Amazon ElastiCache for Redis (cluster mode enabled), TLS enabled	Semantic cache re-enabled only after the <code>distance_threshold</code> bug (see §2) is fixed and validated; RediSearch module availability must be confirmed on ElastiCache (may require self-managed Redis Stack on EKS if module support is unavailable).
<code>.env</code> file	AWS Secrets Manager + External Secrets Operator (syncs into K8s Secrets) + KMS encryption	<code>app/utils.py</code> and <code>app/rag.py</code> continue to read <code>os.getenv(...)</code> ; only the source of those env vars changes (injected by the Secrets Operator, never checked into git).
<code>data/</code> folder mounted into container	Amazon S3 bucket (versioned, encrypted, access-logged) as canonical source-of-truth; ingestion job pulls from S3	Enables document lifecycle management, versioning, and a real approval workflow instead of ad-hoc file drops.
Single FastAPI process (<code>app/api.py</code>)	FastAPI containerized, deployed as an EKS Deployment behind an Ingress/ALB, horizontally scaled via HPA	Endpoints gain an AuthN/Z middleware layer (see §5.1) inserted in front of existing route handlers.
Manual <code>curl -X POST /ingest</code>	Ingestion Kubernetes <code>Job</code> / <code>CronJob</code> triggered via governed pipeline (approval + S3 event or schedule), or an internal admin UI with RBAC	Prevents arbitrary/anonymous ingestion triggers; see §5.1 and §10.

POC Component (current code)	Enterprise Replacement	Notes
Local file logging (<code>app/logger.py</code> , <code>logs/app.log</code>)	Structured JSON logs shipped to CloudWatch Logs / OpenSearch, correlated by trace ID	Same logging calls in code; only the log handler/sink changes (e.g., add a CloudWatch/OpenTelemetry handler).
No CI/CD	GitHub Actions/GitLab CI + Terraform + Helm + image scanning (Trivy/ECR scanning) + GitOps (ArgoCD)	See §9.

4.3. 4.3 Networking

- **VPC** with public subnets (ALB only) and private subnets (EKS nodes, RDS, ElastiCache).
- **Security Groups**: least-privilege — EKS pods → RDS (5432) and ElastiCache (6379) only; no direct internet access from private subnets except via NAT Gateway for approved egress (OpenAI/Cohere APIs) or VPC endpoints where supported.
- **VPC Endpoints (PrivateLink)** for S3, Secrets Manager, KMS, CloudWatch to avoid traffic traversing the public internet.
- **Service Mesh (Istio or AWS App Mesh)** for mTLS between the query service, ingestion service, and any internal microservices.
- **AWS WAF** in front of the ALB to mitigate common web attacks (SQLi, XSS, rate-based rules) and to add basic bot/DDoS protection.

Chapter 5. 5. Security & Compliance (Primary Focus)

5.1. 5.1 Authentication & Authorization

Current gap: `app/api.py` has zero authentication — `/ask`, `/ingest`, `/ingest/status` are fully open.

Target design:

- **Identity Provider Integration:** Enterprise SSO via OIDC (Okta, Azure AD, or AWS IAM Identity Center). All human users authenticate through SSO; the API never handles raw passwords.
- **API Authentication:** All requests to the FastAPI service must present a valid JWT (OIDC access token) validated against the IdP's JWKS endpoint. Implemented as FastAPI middleware/dependency inserted before existing route handlers in `app/api.py` (e.g., `Depends(get_current_user)`).
- **Service-to-Service Authentication:** Non-human callers (batch jobs, other internal services) use OAuth2 client-credentials grants or mTLS certificates issued via the service mesh — never shared static API keys.
- **RBAC/ABAC Model:**

Role	Permissions	Maps to code change
<code>viewer</code> (default employee)	Call <code>/ask</code> ; results filtered to categories the user's group is entitled to (e.g., all-employee: <code>guides</code> , <code>faqs</code> , <code>announcements</code> ; HR-only: <code>handbooks</code> , <code>policies</code>).	<code>app/api.py</code> 's hard-coded <code>'category = None</code> is replaced with a category list resolved from the authenticated user's group claims, then passed into <code>answer_with_docs_async()</code> / <code>_build_chain()</code> in <code>app/rag.py</code> (already supports a single category filter — extend to a list/ <code>IN</code> filter).
<code>content-admin</code>	Trigger <code>/ingest</code> , view <code>/ingest/status</code> , manage document lifecycle (add/retire sources in S3).	New <code>require_role("content-admin")</code> dependency guarding <code>POST /ingest</code> .
<code>platform-admin</code>	Full operational access: view audit logs, manage index rebuilds, rotate secrets (via Secrets Manager, not app-level).	Separate internal admin service/route group, not exposed on the public ALB.

- **IAM Roles for Service Accounts (IRSA):** Each EKS pod (query service, ingestion job) assumes a narrowly scoped IAM role — e.g., ingestion pods can `GetObject` from the S3 documents bucket and read the RDS/embedding secret, but cannot access unrelated AWS resources.

5.2. 5.2 Encryption

In transit:

- TLS 1.2+ terminated at the ALB; end-to-end TLS re-encryption from ALB → Ingress → pods.

- mTLS between all internal services (query service ↔ Postgres, query service ↔ Redis, query service ↔ Cohere/OpenAI egress) via the service mesh sidecar.
- RDS and ElastiCache connections forced to use TLS (`sslmode=require` for Postgres; `rediss://` for ElastiCache), replacing the current unencrypted `postgresql+asyncpg://` / `redis://` local connections.

At rest:

- RDS storage encrypted with a customer-managed KMS key (CMK); automated snapshots inherit encryption.
- ElastiCache encryption-at-rest enabled with KMS.
- S3 bucket (source documents) uses SSE-KMS with a dedicated CMK; bucket versioning + object lock (WORM) for compliance-sensitive document categories (e.g., `policies`, `handbooks`).
- EKS node/EBS volumes encrypted by default via KMS.
- CloudWatch Log Groups encrypted with KMS.

5.3. 5.3 Secrets Management

- Replace `.env/env_file` usage entirely with **AWS Secrets Manager**.
- Use the **External Secrets Operator** (or AWS Secrets & Configuration Provider for CSI) to sync secrets into Kubernetes as native `Secret` objects, mounted as environment variables or files — the application code (`os.getenv("DATABASE_URL")`, `os.getenv("OPENAI_API_KEY")`, etc. in `app/utils.py` and `app/rag.py`) requires **no code changes**, only the secret's origin changes.
- Automated secret rotation for the RDS master password and API keys where the provider supports rotation Lambdas (e.g., RDS credential rotation).
- No secrets are ever committed to git; a pre-commit hook + CI secret-scanning step (e.g., `gitleaks`, `truffleHog`) enforced in the pipeline (§9) to prevent regressions like the current committed `.env` with live keys (explicitly called out as a risk in `1-rag-knowledge-management.adoc`, line ~314).

5.4. 5.4 PII & Data Governance

- **Document classification:** Every document ingested is tagged not only with `category` (existing metadata field) but also a **sensitivity level** (`public`, `internal`, `confidential`, `restricted`) determined during the ingestion approval workflow (§10).
- **PII detection/redaction pipeline:** Before chunking (`app/ingest.py`'s `_chunk()`), run an automated PII-scanning step (e.g., Amazon Comprehend PII detection or Presidio) over extracted text. Detected PII (SSNs, bank account numbers, salary figures not intended for broad retrieval) is either redacted or the document is routed to a restricted-access category requiring elevated RBAC.
- **Data residency:** All ingestion, embedding storage, and logs remain within the enterprise's approved AWS region(s). If a non-US-based LLM/embedding provider is used, a Data Processing Agreement (DPA) and regional endpoint must be validated; otherwise prefer Amazon Bedrock (in-region, in-VPC) over external OpenAI/Cohere calls for regulated categories.

- **Right to erasure / retention:** A documented retention policy per data class (e.g., cached answers purged after 30 days; audit logs retained 1 year; source documents retained per records-management policy) with an operational procedure to delete a specific document's chunks from `langchain_pg_embedding` (by `source` metadata) and its cached responses on request.

5.5.5 Audit Logging

- Every `/ask` request logs (to an **immutable, append-only** store — e.g., CloudWatch Logs with a restrictive IAM policy, or a dedicated audit table with no delete permission granted to the app role): authenticated user ID, timestamp, question text, resolved category filter, returned source document IDs, and response latency.
- Every administrative action (`/ingest` trigger, config change, role assignment) is logged with the actor's identity and a correlation/trace ID.
- CloudTrail enabled account-wide to capture all AWS API-level actions (who read/rotated a secret, who modified an IAM policy, who accessed the S3 documents bucket directly).
- Audit logs are shipped to a security-owned account/log sink (separate from the application account) to prevent tampering by application administrators — a common SOC 2 / segregation-of-duties control.

5.6 LLM-Specific Security

- **Prompt injection mitigation:** Because retrieved document content is "stuffed" into the prompt (`create_stuff_documents_chain` in `app/rag.py`), enterprise documents could theoretically contain adversarial instructions. Mitigate via: input/output guardrails (e.g., a lightweight classifier or rule-based filter checking for instruction-like patterns in retrieved chunks before they reach the LLM), and by keeping the system prompt's instruction-following boundary strict (already partially done via the existing `SYSTEM` prompt in `app/rag.py`).
- **Output filtering:** Scan LLM responses for potential leakage of restricted-category content when a lower-privileged user's query indirectly surfaces sensitive information (defense-in-depth on top of the RBAC category filter in §5.1).
- **Third-party data exposure:** Document that question text + retrieved context is sent to OpenAI/Cohere; ensure enterprise agreements with these vendors include a Data Processing Agreement and opt-out of training on submitted data (OpenAI/Cohere enterprise tiers support this). Evaluate Amazon Bedrock as an in-VPC alternative for the most sensitive categories to avoid third-party egress entirely.

5.7 Compliance Control Mapping (Illustrative)

Framework Control Area	Requirement	Design Element Satisfying It
SOC 2 — Logical Access	Access to systems restricted to authorized users	OIDC SSO + RBAC/ABAC (§5.1), IRSA for service accounts
SOC 2 — Encryption	Data encrypted at rest and in transit	KMS-backed encryption (§5.2), TLS/mTLS everywhere

Framework Control Area	Requirement	Design Element Satisfying It
SOC 2 — Change Management	Changes to production are reviewed and tracked	CI/CD with approval gates, GitOps (§9)
SOC 2 — Monitoring	Security events detected and responded to	GuardDuty, Security Hub, CloudWatch alarms (§7)
GDPR — Data Minimization / Right to Erasure	Personal data processed only as necessary; deletable on request	PII detection/redaction, per-document deletion procedure (§5.4)
GDPR — Data Residency	EU personal data processed within approved jurisdiction	Region-pinned S3/RDS/ElastiCache, Bedrock in-region option (§5.4)
CIS AWS Foundations	Least-privilege IAM, no long-lived credentials where avoidable	IRSA, Secrets Manager rotation, no static keys in <code>.env</code> (§5.1, §5.3)

Chapter 6. 6. Scalability & Reliability

- **Compute:** FastAPI query service deployed as an EKS Deployment with Horizontal Pod Autoscaler (HPA) scaling on CPU/RPS/queue-depth; ingestion runs as a separate Job/CronJob so query-path scaling is unaffected by ingestion load (preserving the existing async-lock design in `app/api.py`'s ``_ingest_lock`, now scoped per-pod with a distributed lock — e.g., a Postgres advisory lock or Redis lock — since multiple replicas mean `asyncio.Lock()` alone is no longer sufficient across pods).
- **Database:** Amazon RDS for PostgreSQL Multi-AZ (synchronous standby) for automatic failover; a read replica serves the RAGAS evaluation job (`app/eval_ragas.py`) and reporting workloads to isolate them from the production query path.
- **Cache:** ElastiCache for Redis in cluster mode with automatic failover across multiple shards/replicas.
- **Vector index scale:** The existing HNSW index (`app/ingest.py`'s ``_create_index()`, `m=16`, `ef_construction=64`) is retained; index-build parameters revisited for larger enterprise-wide corpora (potentially millions of chunks vs. the current ~300).
- **Backups & DR:** Automated RDS snapshots (point-in-time recovery, ≥15-minute RPO), S3 cross-region replication for source documents, periodic disaster-recovery drills validating the ≤1 hour / ≤4 hour RTO targets from §3.2.
- **Multi-AZ by default;** multi-region as a future phase for organizations requiring active-active DR (documented as a roadmap item, §11).

Chapter 7. 7. Observability

- **Structured logging:** Extend `app/logger.py` to emit structured JSON logs (already using Python's `logging` module) shipped to CloudWatch Logs / OpenSearch, tagged with a request/trace ID propagated through `api.py` → `rag.py` → `utils.py`.
- **Metrics:** Expose Prometheus metrics (request count/latency per endpoint, ingestion job duration/chunk counts, LLM token usage and cost, cache hit ratio once re-enabled) scraped by an EKS-native Prometheus/Grafana stack or Amazon Managed Prometheus/Grafana.
- **Tracing:** OpenTelemetry instrumentation across the retrieval → rerank → generation chain; LangSmith (already referenced via `LANGSMITH_API_KEY/LANGSMITH_PROJECT` in the current `.env`) retained as an LLM-specific observability layer for prompt/response inspection and chain debugging.
- **Alerting:** CloudWatch Alarms / Grafana alerting on error-rate spikes, p95 latency breaches, ingestion job failures, and anomalous LLM spend.
- **Dashboards:** SLO dashboard tracking availability, latency, and cost against the targets in §3.2.

Chapter 8. 8. Data & Ingestion Governance

- **Canonical source of truth:** Move from local `data/<category>/` folders to a governed **S3 bucket** with a defined prefix structure mirroring categories (`s3://enterprise-rag-docs/<category>/...`), versioned and access-logged.
- **Approval workflow:** New/updated documents go through a lightweight review (content owner + designated approver) before being marked "ready for ingestion" — preventing the current ad-hoc "drop a file in ``data/``" pattern.
- **Sensitivity tagging:** Each document tagged with `category` (existing) **and** `sensitivity` (new — §5.4) at upload time; ingestion (`app/ingest.py`) propagates both into `langchain_metadata/category` columns already present in the `langchain_pg_embedding` schema (`init-db/init.sql`).
- **Versioning & re-ingestion:** Superseded documents are retired (not silently overwritten) so the LLM's existing "prefer the most recent policy by effective date" rule (already encoded in `app/rag.py`'s `PROMPT`) has reliable metadata (effective/expiry dates) to reason over.
- **Category-to-role mapping:** A configuration table (not hard-coded in `api.py`) mapping each `category` to the IdP group(s)/roles authorized to query it — replacing the current single hard-coded `category = None` behavior with a data-driven, auditable mapping.

Chapter 9. 9. CI/CD & Environment Strategy

- **Environments:** `dev` → `staging` → `prod`, each an isolated AWS account or at minimum isolated VPC/namespace, provisioned via **Terraform** (IaC) for all AWS resources (VPC, EKS, RDS, ElastiCache, S3, IAM, Secrets Manager).
- **Application delivery:** Container images built from the existing `Dockerfile`, scanned (Trivy/ECR image scanning + `gitleaks` secret scan) in CI, pushed to Amazon ECR, deployed via **GitOps** (**ArgoCD/Flux**) applying Helm charts to EKS.
- **Dependency management:** Consolidate the scattered `ALL-ENV/requirements.txt` files and the missing `requirements.txt` referenced by the `Dockerfile` into a single, pinned, SCA-scanned `requirements.txt/pyproject.toml` per the note already flagged in `2-rag-knowledge-management.adoc` §8.
- **Testing gates:** Unit tests for `app/ingest.py/app/rag.py/app/utils.py`, integration tests against an ephemeral Postgres+pgvector test container, and a RAGAS regression check (reusing `app/eval_ragas.py` and `seed/qna_test.json`) run in CI before promotion to `staging`.
- **Progressive delivery:** Canary or blue-green rollout for the query service in `prod`, with automatic rollback on SLO alarm breach (§7).

Chapter 10. 10. Migration Roadmap

Phase	Focus	Key Deliverables
Phase 0 — Hardening (weeks 1-3)	Stop-the-bleeding security fixes on the existing POC	Remove live API keys from <code>.env</code> /git history; move secrets to Secrets Manager even before AWS migration; add basic auth (API key or OIDC) in front of <code>/ingest</code> and <code>/ask</code> ; fix or permanently retire the Redis semantic cache <code>distance_threshold</code> bug; create the missing <code>requirements.txt</code> .
Phase 1 — AWS Foundation (weeks 3-8)	Terraform-provisioned VPC, EKS, RDS (Multi-AZ), ElastiCache, S3, Secrets Manager, KMS	Containerized app deployed to EKS dev environment; database migrated from local Postgres/Docker volume to RDS with pgvector; source documents migrated from <code>data/</code> to S3.
Phase 2 — Security & Compliance (weeks 6-12, overlaps Phase 1)	OIDC SSO integration, RBAC/ABAC category mapping, PII scanning pipeline, audit logging	Category-based access control replacing hard-coded <code>category=None</code> ; PII detection integrated into <code>_chunk()</code> /ingestion; immutable audit log store live; compliance control mapping (§5.7) reviewed with security/compliance team.
Phase 3 — Reliability & Observability (weeks 10-14)	HA validation, DR drills, metrics/tracing/alerting rollout	Multi-AZ failover tested; backup/restore drill completed meeting RPO/RTO targets; Grafana/CloudWatch dashboards and alerts live.
Phase 4 — Scale-Out & Governance (weeks 12-18)	CI/CD maturity, ingestion governance workflow, cost controls	GitOps pipeline with progressive delivery; document approval workflow live; per-department query quotas/rate limiting; RAGAS regression gating in CI.
Phase 5 — Continuous Improvement (ongoing)	Multi-region DR evaluation, Bedrock migration evaluation, multi-tenant readiness assessment	Documented decision on whether multi-region/active-active DR or multi-tenant support is justified by enterprise growth (see §12 for extension notes).

Chapter 11. 11. Cost Management

- **Token/LLM cost control:** Re-enable semantic caching (Redis/ElastiCache) once the `distance_threshold` correctness bug is fixed and validated against a diverse regression set (extending `seed/qna_test.json`), to reduce redundant OpenAI/Cohere calls.
- **Per-department quotas:** Rate-limit `/ask` per authenticated user/department (API Gateway or ingress-level rate limiting) to bound worst-case monthly spend.
- **Right-sized infrastructure:** HPA min/max replica bounds tuned to actual enterprise-wide query volume; RDS/ElastiCache instance classes sized from real usage metrics rather than over-provisioned upfront.
- **Cost observability:** Tag all AWS resources by environment/team; integrate AWS Cost Explorer/Budgets alerts alongside the LLM-specific token-usage metrics from §7.
- **Model selection tradeoffs:** Continue using `gpt-4o-mini` (already selected in `app/rag.py` for cost-effectiveness) for general queries; evaluate Amazon Bedrock models as a potentially cheaper/in-VPC alternative for high-volume or highly regulated categories.

Chapter 12. 12. MLOps / LLMOps

This section addresses model governance, evaluation, prompt versioning, and guardrails for the LLM/embedding components — an increasingly critical concern for enterprise RAG systems where model behavior, cost, and quality must be actively managed rather than left to a single hard-coded chain configuration.

12.1. 12.1 Model & Prompt Governance

- **Model registry / approved model list:** Maintain a governed catalog of approved models for each role in the pipeline — embedding model (`text-embedding-3-small`, `app/utils.py`), generation model (`gpt-4o-mini`, `app/rag.py`), and reranker (`rerank-multilingual-v3.0`, `app/rag.py`). Model upgrades (e.g., moving to a newer embedding model) must go through a re-evaluation gate (§12.2) before promotion, since changing the embedding model requires re-embedding the entire `langchain_pg_embedding` corpus.
- **Prompt versioning:** The `SYSTEM` prompt and `PROMPT` template in `app/rag.py` are currently hard-coded in source. Enterprise design externalizes prompts into a version-controlled prompt registry (e.g., LangSmith Prompt Hub, or a Git-backed prompt store) with semantic versioning (`system-prompt v1.3`), change history, and the ability to roll back a regressive prompt change without a full application redeploy.
- **Change approval workflow:** Any change to the system prompt, retrieval `k`, rerank `top_n`, chunking parameters (`chunk_size=900`, `chunk_overlap=120` in `app/ingest.py`), or model selection is treated as a governed change — reviewed and evaluated against the regression suite (§12.2) before merging, similar to a database migration.
- **Model/version metadata on every response:** Every `/ask` response (and its audit log entry, §5.5) records which embedding model, generation model, reranker model/version, and prompt version produced the answer — enabling root-cause analysis when answer quality regresses after a change.

12.2. 12.2 Evaluation & Continuous Quality Monitoring

- **Automated regression evaluation:** `app/eval_ragas.py` already implements RAGAS-based evaluation (faithfulness, answer relevancy, context precision, context recall) against `seed/qna_test.json`. In the enterprise design, this becomes a **CI/CD quality gate** (§9) — no prompt, model, or retrieval-parameter change is promoted to `staging/prod` unless RAGAS scores meet or exceed a defined baseline threshold.
- **Expanded golden dataset:** Grow `seed/qna_test.json` from 2 sample Q&A pairs into a representative, versioned golden dataset covering all document categories (`announcements`, `faqs`, `guides`, `handbooks`, `policies`), including edge cases (ambiguous questions, conflicting-policy questions the system prompt is designed to resolve, and adversarial/out-of-scope questions that should trigger "I don't know").
- **Production quality monitoring:** Beyond pre-release CI gating, sample a percentage of live `/ask` traffic for ongoing RAGAS-style scoring (or LLM-as-judge scoring) to detect quality drift caused by upstream document changes, embedding drift, or model provider updates — surfaced on the

observability dashboards (§7).

- **Human-in-the-loop feedback:** Capture thumbs-up/down or correction feedback from the frontend UI (`app/static/index.html`) per answer, feeding a continuous improvement loop and flagging low-confidence or disputed answers for review by content owners.
- **A/B / shadow evaluation:** Before fully cutting over a new model or prompt version, run it in shadow mode (compute an answer but don't serve it) or A/B test against a subset of users, comparing RAGAS/human-feedback metrics against the current production baseline.

12.3. 12.3 Guardrails

- **Input guardrails:** Validate/sanitize user questions before they reach the retrieval pipeline (e.g., reject empty/oversized inputs, detect and block obvious prompt-injection attempts such as "ignore previous instructions") — implemented as a pre-processing step in `app/api.py`'s `/ask` handler, ahead of `answer_with_docs_async()`.
- **Retrieved-context guardrails:** As referenced in §5.6, scan retrieved chunks for embedded instruction-like content before they are "stuffed" into the prompt by `create_stuff_documents_chain`, reducing the risk of indirect prompt injection via poisoned or adversarial source documents.
- **Output guardrails:** Post-process LLM responses to catch policy violations — e.g., leakage of restricted-category content, hallucinated citations (cross-check `sources` returned by `answer_with_docs_async()` against the actual retrieved `contexts`), toxic/inappropriate language, or responses that ignore the "I don't know" instruction when context is insufficient.
- **Groundedness/faithfulness check:** Optionally run a lightweight automated faithfulness check (reusing the RAGAS faithfulness metric or a cheaper heuristic) synchronously on a sample of responses, or asynchronously on all responses, to detect hallucination in near-real time and feed alerts (§7) when faithfulness drops below threshold.
- **Cost/abuse guardrails:** Rate limiting and per-user/department quotas (§11) double as a guardrail against runaway or abusive usage patterns driving unbounded LLM spend.

12.4. 12.4 LLMOps Tooling Summary

Capability	Tooling Approach
Prompt/version registry	LangSmith Prompt Hub (already integrated via <code>LANGSMITH_API_KEY/LANGSMITH_PROJECT</code>) or a Git-backed prompt store with CI validation
Evaluation harness	RAGAS (<code>app/eval_ragas.py</code> , existing) extended with an expanded golden dataset, wired into CI as a quality gate (§9)
Tracing / debugging	LangSmith tracing across the retrieval → rerank → generation chain, correlated with the audit log (§5.5)
Production monitoring	Sampled live-traffic scoring + human feedback capture, surfaced via observability dashboards (§7)

Capability	Tooling Approach
Guardrail enforcement	Input/output/context guardrail middleware around <code>answer_with_docs_async()</code> in <code>app/rag.py</code> , plus rate limiting at the ingress/API Gateway layer

Chapter 13. 13. Extension Notes: Future Multi-Tenancy

Although this revision targets a **single-tenant enterprise deployment**, the design choices below keep a future multi-tenant evolution low-cost if the organization later needs to serve multiple business units with isolated data:

- The `category/sensitivity` metadata model (§5.4, §8) can be extended with a `tenant_id` column in `langchain_pg_embedding` with minimal schema change.
- RBAC/ABAC (§5.1) already resolves authorization from IdP group claims — adding a tenant claim is a natural extension rather than a redesign.
- S3 prefix-per-category structure (§8) can become S3 prefix-per-tenant-per-category.
- EKS namespace-per-tenant with network policies could replace the current single-namespace deployment if strict compute isolation is later required.

Chapter 14. 14. Open Questions / Risks

- **Redis semantic cache:** Should it be permanently retired in favor of a simpler deterministic (non-semantic) cache, or fixed with a stricter, well-tested `distance_threshold`? This decision affects both cost (§11) and correctness risk.
- **RedisSearch on ElastiCache:** Confirm whether Amazon ElastiCache supports the RedisSearch module required by `RedisSemanticCache`; if not, a self-managed Redis Stack on EKS (StatefulSet) may be required instead of ElastiCache.
- **LLM provider strategy:** Continue with OpenAI + Cohere (external SaaS, requires DPA/data-egress review) vs. migrate to Amazon Bedrock (in-VPC, potentially simpler compliance story but requires model/quality re-validation via `app/eval_ragas.py`).
- **PII scanning tool selection:** Amazon Comprehend (managed, pay-per-use) vs. self-hosted Microsoft Presidio (more control, more operational overhead) — needs a proof-of-concept against real sample documents (`data/handbooks/employee_handbook_2025.docx`, `data/policies/*.pdf`).
- **Approval workflow tooling:** Build a lightweight internal admin UI vs. integrate with an existing enterprise content-management/workflow tool (e.g., ServiceNow, SharePoint approval flows) for the document governance process in §8.
- **Regulatory scope confirmation:** Confirm with Legal/Compliance which specific frameworks apply (SOC 2, GDPR, HIPAA, industry-specific) so the control mapping in §5.7 can be finalized and formally audited.